

```
<!--Análisis De Algoritmos-->
```

Lowest Common Ancestor {

```
<Por="
```

```
Ismael Gándara Cornejo,
```

```
Ángel Miguel Villalvazo Vázquez
```

```
"/>
```

```
}
```



Contenidos

- 01 Introducción
- 02 Código explicado
- 03 Aplicaciones en la vida real
- 04 Complejidad temporal
- 05 Complejidad espacial
- 06 Otras técnicas
- 07 Conclusión

Introducción {

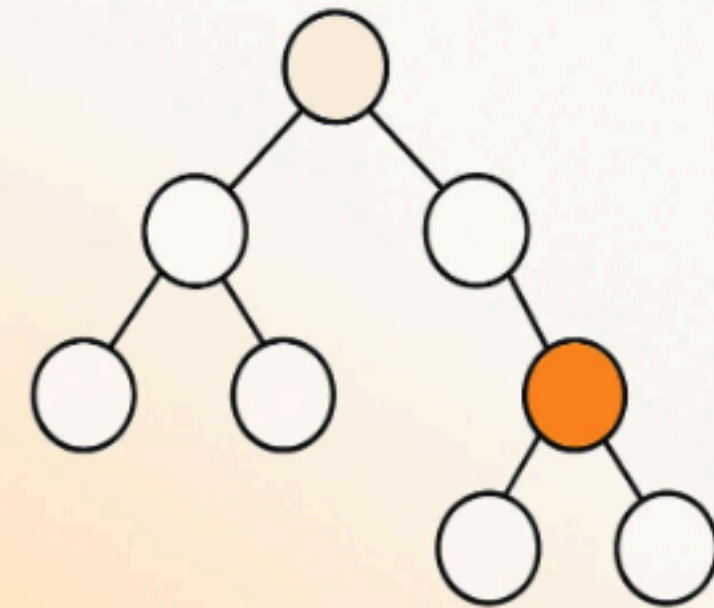
¿De qué se trata el algoritmo?

Importancia

Pedagógica & Eficiencia:

- Simple y útil
- Lenta en el peor caso

Lowest Common Ancestor in a Binary Tree



Código Explicado{

2

1

```
# Definición de nodos
class Node: 7 usages
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

3

```
# Buscamos en el subárbol izquierdo Y derecho
if ((root.left is not None and find_path(root.left, path, k)) or
    (root.right is not None and find_path(root.right, path, k))):
    return True

path.pop()
return False
```

```
# Función para encontrar raíz a nodo
def find_path(root, path, k): 4 usages

    # Caso base
    if root is None:
        return False

    # Agrega
    path.append(root.key)

    # Ruta completa
    if root.key == k:
        return True
```

}

4

Código Explicado{

4

```
# Función principal para encontrar el LCA
def find_lca(root, n1, n2):
    """
    Encuentra el Ancestro Común más Bajo (LCA) de los nodos n1 y n2.
    """
    # rutas n1,n2
    path1 = []
    path2 = []
```

5

```
# Obtenemos las rutas
if not find_path(root, path1, n1) or not find_path(root, path2, n2):
    return -1
```

6

```
while i < len(path1) and i < len(path2):
    if path1[i] != path2[i]:
        break
    i += 1
return path1[i - 1]

root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
root.right.left = Node(6)
root.right.right = Node(7)
"""
    1
   2  3
  4 5 6 7"""
```

}

Código Explicado{

7

```
# Buscamos el LCA de los nodos 4 y 5
lca = find_lca(root, n1: 4, n2: 5)
if lca != -1:
    print(f"El LCA de 4 y 5 es {lca}")
else:
    print("Al menos uno de los nodos no está en el árbol.")

# Buscamos el LCA de los nodos 4 y 6
lca = find_lca(root, n1: 4, n2: 6)
if lca != -1:
    print(f"El LCA de 4 y 6 es {lca}")
else:
    print("Al menos uno de los nodos no está en el árbol.")
```

Aplicaciones en la vida real {

Control de Versiones

Modela el historial de un proyecto como un Grafo Acíclico Dirigido (DAG), donde cada commit es un nodo y las relaciones padre-hijo conectan los commits a lo largo del tiempo.

En estos sistemas donde hay operaciones de fusión de archivos, Git debe encontrar un punto de partida común para reconciliar las diferencias.

Con el comando `git merge-base` inicia un proceso "fusión de tres vías" con tres puntos de datos del estado del código en:

1. El LCA (la base de la fusión).
2. La punta de la primera rama.
3. La punta de la segunda rama.

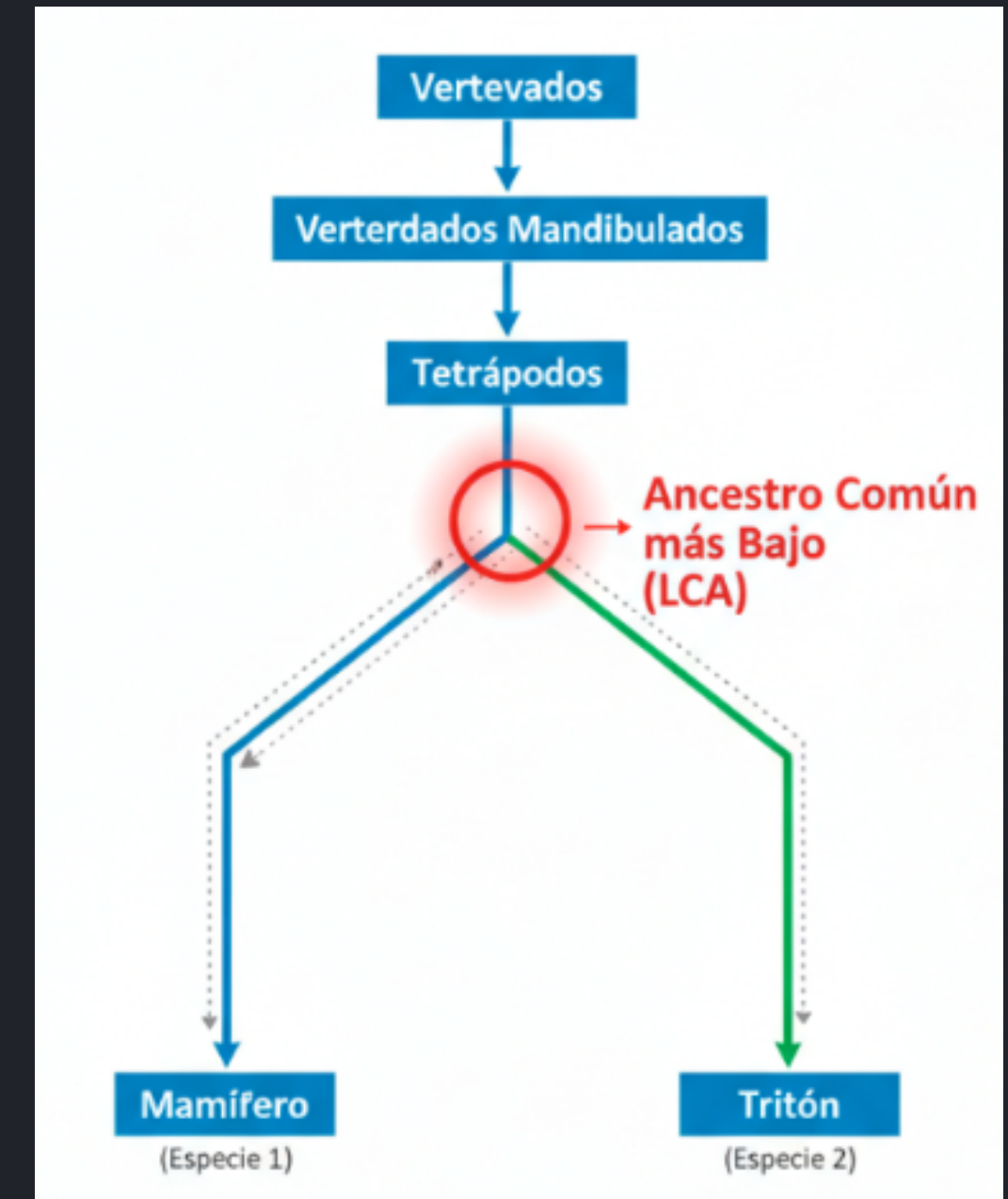
Git compara los cambios de la base a cada punta para determinar cómo combinar las modificaciones de manera inteligente.



Relaciones Evolutivas en Árboles Filogenéticos

En biología computacional se usan estos árboles donde las hojas representan especies u organismos existentes, mientras que los nodos internos representan ancestros comunes de los que divergieron sus descendientes.

Encontrar el LCA es equivalente a identificar a un ancestro común más reciente. Esta es una operación fundamental para los biólogos evolutivos, ya que les permite rastrear la herencia de rasgos genéticos y comprender los patrones de especiación.



Complejidad Temporal {

- **Método de Intersección de Rutas:** Encuentra los dos nodos en el árbol partiendo desde la raíz. Tiene una complejidad de $O(N)$ pero como se realiza dos veces con una comparación de rutas de longitud máxima H , la complejidad total es de $O(N)+O(N)+O(H)$, que igualmente se simplifica a $O(N)$.
- **Método de Recorrido de Ancestros:** Sube desde cada nodo objetivo hacia la raíz con punteros a padres. El tiempo necesario es proporcional a la profundidad del nodo, por lo que los dos recorridos toman $O(H)$ tiempo.

Un árbol perfectamente equilibrado tiene altura H de aproximadamente $\log_2(N)$. En cambio, en un árbol degenerado, H es aproximadamente N .



}

Complejidad Espacial {

Método de Intersección de Rutas

El espacio requerido es el almacenamiento de las dos rutas desde la raíz. La longitud máxima de cada ruta es la altura del árbol, H . Por lo tanto, la complejidad espacial es $O(H)$.

Método de Recorrido de Ancestros

El espacio adicional principal es el conjunto hash utilizado para almacenar los ancestros del primer nodo. En el peor de los casos, este conjunto igual contendrá H elementos con la complejidad de $O(H)$.

}



Técnicas Más Eficientes {

Levantamiento Binario (Binary Lifting): El Salto Logarítmico

Programación dinámica con preprocesamiento.

Con un único recorrido de todos los nodos calcula los padres inmediatos y profundidades para luego construir una tabla de ancestros almacenandolos a distancias de potencias de dos. Ahí, para encontrar el LCA de dos nodos u y v , ambos nodos se elevan a una potencia simultáneamente, encontrando el padre del ancestro más alto de u que no es un ancestro de v .

Por ejemplo, si se quiere el 13er ancestro, se puede saltar 2^3 y luego 2^2 , sumando $8+4=12$. Este proceso toma tiempo $O(\log N)$.



Recorrido De Euler

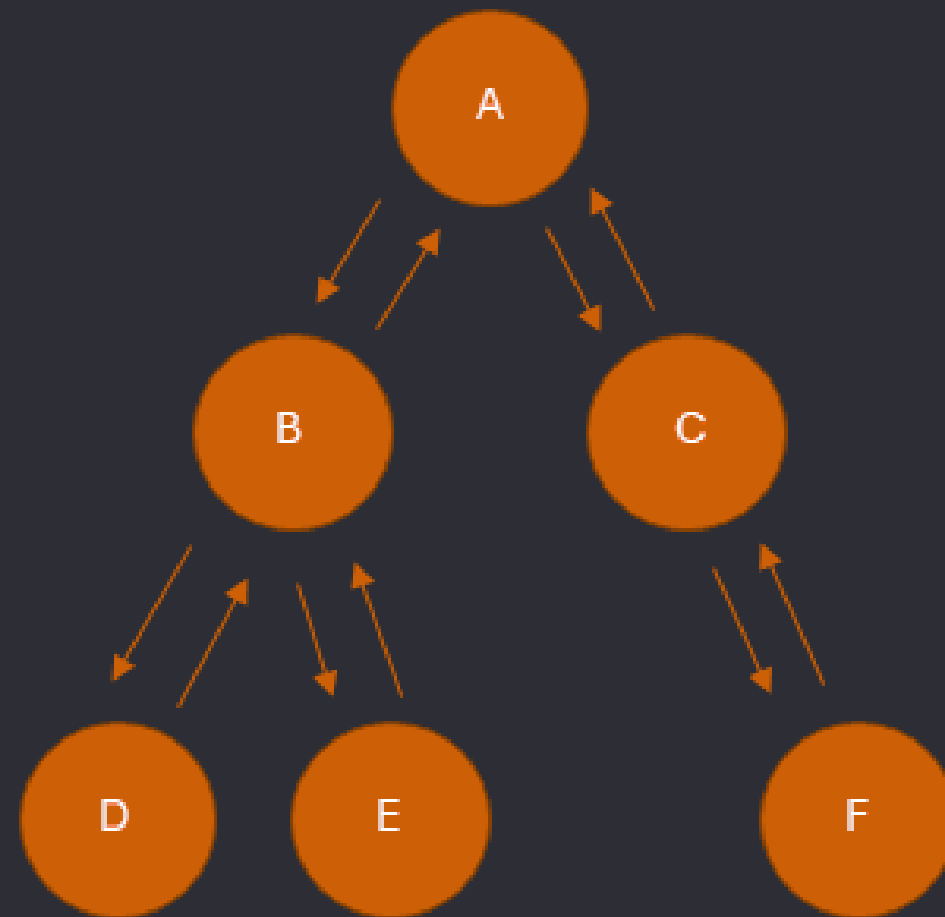
Recorrido DFS que registra un nodo cada vez que se visita al descender y cuando se regresa a él. Esto genera un arreglo de $2N-1$ nodos y después crea un segundo arreglo que almacena la profundidad de cada nodo en el orden del Recorrido de Euler.

Por ejemplo, si una lista de nodos es [A, B, D, B, E, B, A, C, F, C, A]

Y las profundidades son (A=0, B=1, C=1, D=2, E=2, F=2)

La lista de profundidades sería: [0, 1, 2, 1, 2, 1, 0, 1, 2, 1, 0]

Para el LCA de u y v, primero se localiza la primera vez que aparecen u y v en la lista del recorrido. Luego, se mira la sub-lista de profundidades con números y en el rango que está entre lo que localizamos, el número menor es el LCA.



}



Conclusión {

Algoritmo

La elección de un algoritmo de LCA debe ser una decisión basada en los requisitos específicos de la aplicación. El uso del algoritmo LCA de fuerza bruta no es aconsejable en los siguientes casos:

- Cuando el número de consultas es alto.
- En aplicaciones de rendimiento crítico donde un tiempo de consulta de $O(N)$ o $O(H)$ constituye un cuello de botella.
- Para árboles muy grandes.

Limitaciones

Con cada consulta se recalculan rutas desde la raíz, sin memoria de cálculos anteriores. Podría mejorarse con un preprocesamiento, como almacenar en caché o precalcular información sobre los ancestros para evitar esta repetición.

Los algoritmos como el Levantamiento Binario y el recorrido de Euler abordan esta ineficiencia al invertir cómputo por adelantado para obtener consultas rápidas.

```
<!--Fin de la presentación-->
```

Gracias {

```
<Algoritmo="Lowest Common Ancestor"/>
```

}